The **ASCII values** are:

- `'0'` → **48**
- `'9'` → **57**

Quick fact

Digits `'0'` to `'9'` are **continuous** in ASCII:

arduino Copy code

```
'0' = 48
'1' = 49
...
'9' = 57
```

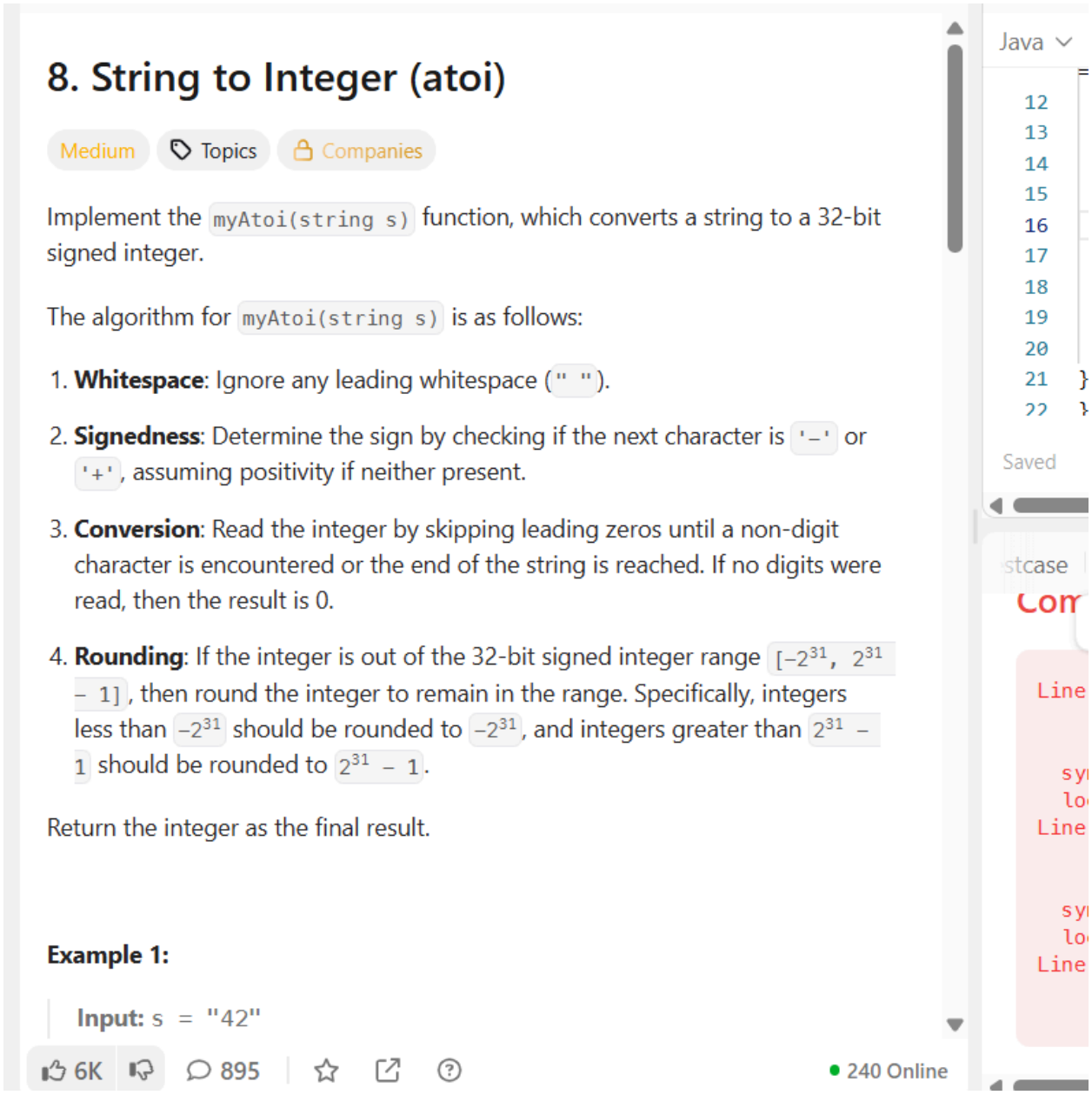
Useful Java tip

The **ASCII value of** `'a'` is:

👉 `'a'` **= 97**

Related values (useful to remember)

- `'A'` = 65
- `'a'` = 97
- `'z'` = 122



8. String to Integer (atoi)

Medium | Topics | Companies

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace**: Ignore any leading whitespace (`" "`).
2. **Signedness**: Determine the sign by checking if the next character is `'-'` or `'+'`, assuming positivity if neither present.
3. **Conversion**: Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. **Rounding**: If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31}, and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

Example 1:

> **Input:** s = "42"

6K 895 240 Online

Java

Saved

2 `num = num * 10 + (s.charAt(i) - '0');`

This line **builds the number digit by digit**.

3️⃣ Step-by-step Dry Run

Input:

arduino Copy code

```
"345"
```

Initial:

ini Copy code

```
num = 0
```

Character	Calculation	num
'3'	0*10 + 3	3
'4'	3*10 + 4	34
'5'	34*10 + 5	345

4️⃣ Why this logic is correct

2️⃣ ASCII values matter

Character	ASCII value
`'0'`	48
`'1'`	49
`'5'`	53
`'9'`	57

So when you write:

→ entire string scanned once.

✔ **Time Complexity = O(n)**

💾 Space Complexity

O(1) (Constant Space)

- Uses only a **fixed number of variables**:
  - `i`, `n`, `num`, `sign`
- No extra data structures (arrays, lists, stacks)

✔ **Space Complexity = O(1)**

```java
class Solution {
    public int myAtoi(String s) {
        int n = s.length();
        int i=0;
        long num =0;
        //skipping white spaces
        while(i<n &&s.charAt(i)==' '){
            i++;
        }
        int sign =1;
        //checking sign
      if(i<n && (s.charAt(i)=='-'||s.charAt(i)=='+')){
sign = (s.charAt(i)=='-')?-1:1;
i++;
      }
      //converting into digit
while(i<n && (s.charAt(i)>='0' && s.charAt(i)<='9')){
    num = num*10+(s.charAt(i)-'0');
    //handling overflow
    if(num*sign>Integer.MAX_VALUE){
      return Integer.MAX_VALUE;
    }
     if(num*sign<Integer.MIN_VALUE){
      return Integer.MIN_VALUE;
    }
    i++;
}
return (int)(num*sign);
    }
```

50. Pow(x, n)

Medium | Topics | Companies

Implement pow(x, n), which calculates `x` raised to the power `n` (i.e., x^n).

Example 1:

```
Input: x = 2.00000, n = 10
Output: 1024.00000
```

Example 2:

```
Input: x = 2.10000, n = 3
Output: 9.26100
```

Example 3:

```
Input: x = 2.00000, n = -2
Output: 0.25000
Explanation: 2^-2 = 1/2^2 = 1/4 = 0.25
```

Constraints:

- `-100.0 < x < 100.0`

2️⃣ Simple Recursive Java Code (Brute Force)

java

Copy code

```java
class Solution {
    public double myPow(double x, int n) {
        if (n == 0) return 1.0;

        if (n < 0)
            return 1.0 / myPow(x, -n);

        return x * myPow(x, n - 1);
    }
}
```

Call stack:

Copy code

```
myPow(2,4)
= 2 * myPow(2,3)
= 2 * 2 * myPow(2,2)
= 2 * 2 * 2 * myPow(2,1)
= 2 * 2 * 2 * 2 * myPow(2,0)
= 2 * 2 * 2 * 2 * 1
= 16
```

4️⃣ Time & Space Complexity

- **Time:** `O(n)`
- **Space:** `O(n)` (recursive stack)

```
1  class Solution {
2      public double fastmyPow(double x,long n){
3          if(n==0)return 1.0;
4          double half = fastmyPow(x,n/2);
5          if(n%2==0){
6  return half*half;
7          }
8          else{
9              return half*half*x;
10         }
11     }
12     public double myPow(double x, int n) {
13 long N = n;//to remove overflow
14 
15     if(N<0){
16        x=1/x;
17        N=-N;
18     }
19     return fastmyPow(x,N);
20     }
21 }
```

optimal solution

ChatGPT Free offer × Share ···

Rules used

1. If n is even

$$x^n = (x^2)^{n/2}$$

2. If n is odd

$$x^n = x \cdot x^{n-1}$$

By repeatedly halving n, we get a logarithmic algorithm.

Why is it optimal?

- **Time Complexity:**

$$O(\log n)$$

because n is divided by 2 at each step.

- **Space Complexity:**

$$O(\log n)$$

due to the recursion stack (can be reduced to $O(1)$ using iteration).

leetcode

In competitive programming and algorithms, you'll often see this line:

> **"Since the answer may be large, return it modulo $10^9 + 7$"**

Let's break it down clearly and then see **how to do it in Java**.

---

1. What is $10^9 + 7$?

$$10^9 + 7 = 1{,}000{,}000{,}007$$

It is:

- A **large prime number**
- Commonly used as a **modulus** in programming problems

Why this number?

- Keeps numbers **within integer limits**
- Prime modulus helps with **division, inverses, and modular arithmetic**
- Fits safely inside **32-bit int**, but calculations usually use `long`

1. What is $10^9 + 7$?

$$10^9 + 7 = 1{,}000{,}000{,}007$$

It is:

- A **large prime number**
- Commonly used as a **modulus** in programming problems

Why this number?

- Keeps numbers **within integer limits**
- Prime modulus helps with **division, inverses, and modular arithmetic**
- Fits safely inside **32-bit int**, but calculations usually use `long`

---

2. What does "return answer modulo $10^9 + 7$" mean?

Instead of returning the full (very large) answer, you return:

$$\text{answer} \bmod 1{,}000{,}000{,}007$$

Example:

```java
Actual answer = 12345678912345
Returned answer = 12345678912345 % 1000000007
```

Free offer ×

3. Key rules of modulo arithmetic

For any modulus `MOD`:

- **Addition**

$$(a + b) \bmod MOD = ((a \bmod MOD) + (b \bmod MOD)) \bmod MOD$$

- **Multiplication**

$$(a \times b) \bmod MOD = ((a \bmod MOD) \times (b \bmod MOD)) \bmod MOD$$

⚠️ Always apply `% MOD` **during** computation, not only at the end.

---

3. When do we use `static` then?

Use `static` when:

- You want to call the method **without creating an object**
- You are calling it from `main()`

Example:

java Copy code

```java
class Test {
    public static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println(add(3, 4)); // no object needed
    }
}
```

Brute Code (Correct & Safe)

java

Copy code

```java
class Solution {
    static final int MOD = 1000000007;

    public int countGoodNumbers(long n) {
        long ans = 1;

        for (long i = 0; i < n; i++) {
            if (i % 2 == 0) {
                ans = (ans * 5) % MOD;  // even index
            } else {
                ans = (ans * 4) % MOD;  // odd index
            }
        }

        return (int) ans;
    }
}
```

Why this is truly "brute force"

Aspect	Value
Time complexity	O(n)
Space	O(1)
Logic	Direct simulation
Optimization	❌ None

Important Notes ⚠️

- Works fine for **small** `n` ($\leq 10^6$)
- Will **TLE** for large `n` (up to 10^{15} in LeetCode)
- No recursion → no stack overflow
- Modulo applied correctly

Explanation:

4 in binary is **100**.

To make it **4 bits**, we add a leading zero → **0100**.

The number **5** in **4-bit binary** is:

0101

Explanation:

$5_{10} = 101_2$

Add a leading zero to make it 4 bits → **0101**

Why this works (intuition)

We are using this identity repeatedly:

$$a^n = \begin{cases} a \times a^{n-1} & n \text{ odd} \\ (a^2)^{n/2} & n \text{ even} \end{cases}$$

e **why** `exp >>= 1` **makes the exponent half.**

4. Why does this always divide by 2?

In binary:

- Each position represents a **power of 2**
- Shifting right by 1:
  - Removes the **least significant bit**
  - Moves every bit one place to the right

That is mathematically equivalent to:

$$\left\lfloor \frac{\text{number}}{2} \right\rfloor$$

Description | Accepted | Editorial | Solutions | Submissions

Medium | Topics | Companies | Hint

A digit string is **good** if the digits **(0-indexed)** at **even** indices are **even** and the digits at **odd** indices are **prime** (`2`, `3`, `5`, or `7`).

- For example, `"2582"` is good because the digits (`2` and `8`) at even positions are even and the digits (`5` and `2`) at odd positions are prime. However, `"3245"` is **not** good because `3` is at an even index but is not even.

Given an integer `n`, return *the **total** number of good digit strings of length* `n`. Since the answer may be large, **return it modulo** $10^9 + 7$.

A **digit string** is a string consisting of digits `0` through `9` that may contain leading zeros.

Example 1:

```
Input: n = 1
Output: 5
Explanation: The good numbers of length 1 are "0", "2", "4", "6", "8".
```

Example 2:

```
Input: n = 4
Output: 400
```

Example 3:

2.3K | 199 | 47 Online

```java
class Solution {
    static final int mod = 1000000007;
    public long calculatepow(long base,long exponent){
        long res = 1;
 base = base%mod;
long e = exponent;
while(e>0){
    if((e&1)==1){
        // e is odd
        res = (res*base)%mod;
    }
    base = (base*base)%mod;
    e>>=1;//this will reduce e to half
}
return res;
    }
    public int countGoodNumbers(long n) {
        long evencount = (n+1)/2;//5 options
              long oddcount = (n)/2;//4 optios
              long ans = (calculatepow(5,evencount)*calculatepow(4,oddcount))%mod;
              return (int)ans;
    }
}
```

leetcode chatgpt

Got it — **JAVA** 👍

Here is a **Java program using recursion to print natural numbers (1 to n)**:

java Copy code

```java
import java.util.Scanner;

public class Main {

    static void printNatural(int n) {
        if (n == 0)
            return;
        printNatural(n - 1);
        System.out.print(n + " ");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();

        printNatural(n);
    }
}
```

printig numbers from n to 1

```java
import java.util.*;
public class Main
{
    public static void print(int n){
        if(n==0)return;
        System.out.print(n+" ");
        print(n-1);
    }
    public static void main(String[] args) {
        System.out.println("Hello World");
        print(5);
    }
}
```

Recursive calls:

```scss
printNatural(n)
printNatural(n-1)
printNatural(n-2)
...
printNatural(1)
printNatural(0)
```

Number of calls = n + 1

👉 So, **stack space used = O(n)**

3️⃣ Why space is O(n)

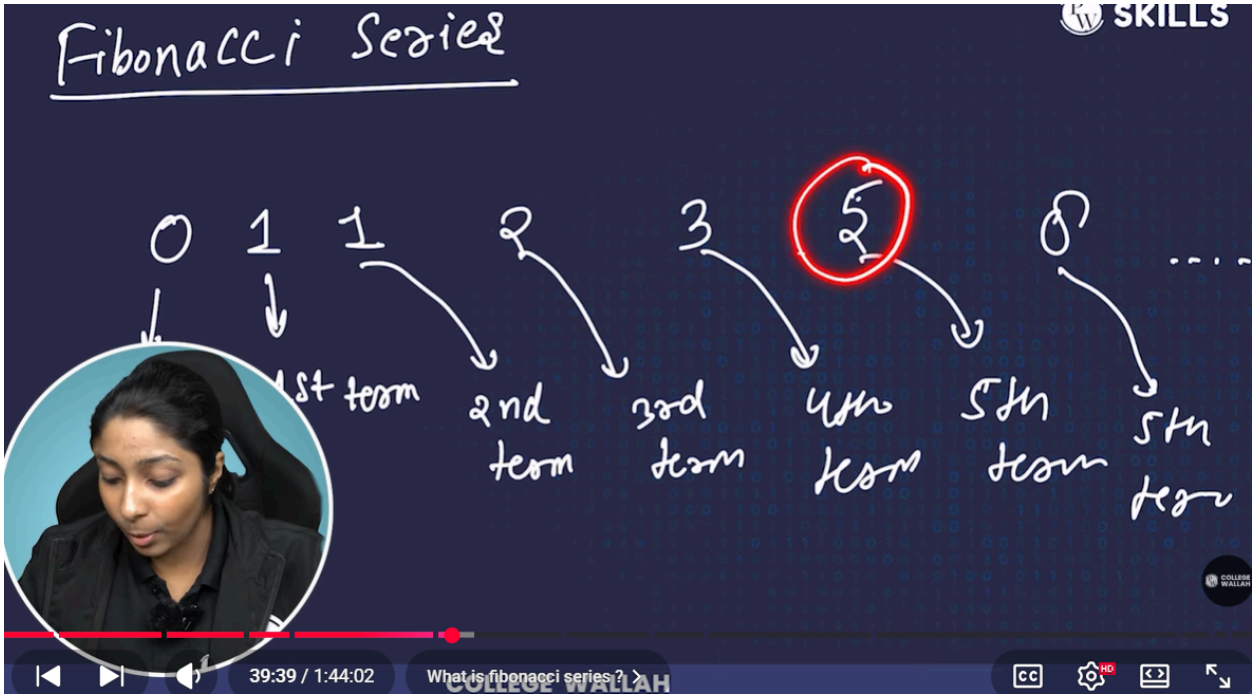
- Every recursive call waits in memory until base case is reached
- Calls are removed **only while returning**
- Maximum stack depth =

finding factorial of a number

```java
import java.util.*;
public class Main{
    public static int factorial(int n){
        if(n==1)return 1;
        int ans = n*factorial(n-1);
        return ans;
    }
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("type n for factorial:");
        int n = sc.nextInt();
        System.out.print("factorial of n:"+factorial(n));
    }
}
```

input

```
type n for factorial:
5
factorial of n:120
```



1st and 2nd terms are fixed i.e 0 and 1 and other left nos are sum of prev numbers

```java
public class Main {

    // Recursive function to print first n Fibonacci numbers
    public static void printFibonacci(int n, int a, int b) {
        if(n == 0) return; // Base case: no numbers to print
        System.out.print(a + " "); // Print current number
        printFibonacci(n - 1, b, a + b); // Recursive call with next
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n for Fibonacci series: ");
        int n = sc.nextInt();
        printFibonacci(n, 0, 1); // Start with 0 and 1
    }
}
```

sum of fibonacci seris

ava

Copy code

```java
import java.util.Scanner;

public class Main {

    // Recursive function to find n-th Fibonacci number
    public static int fibonacci(int n) {
        if(n == 0) return 0;
        if(n == 1) return 1;
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();

        int sum = fibonacci(n + 2) - 1; // Using formula
        System.out.println("Sum of first " + n + " Fibonacci numbers
    }
}
```

1. The formula

The sum of the first n Fibonacci numbers:

$$S_n = F_0 + F_1 + F_2 + \cdots + F_{n-1} = F_{n+1} - 1$$

Some books write $F(n+2) - 1$ if you start counting from $F_1 = 1$. Either way, the idea is:

$$\text{Sum of first n Fibonacci numbers} = F_{n+2} - 1$$

So, the main task reduces to **finding** F_{n+2}.

keep in mind how to store list in a list

✅ Case 1: You want to store a list `[2, 3]` inside another ArrayList

That means: **ArrayList of ArrayLists**.

java Copy code

```java
import java.util.*;

ArrayList<ArrayList<Integer>> list = new ArrayList<>();

ArrayList<Integer> inner = new ArrayList<>();
inner.add(2);
inner.add(3);

list.add(inner);

System.out.println(list);   // [[2, 3]]
```

shortcut

Shorter way

```java
list.add(new ArrayList<>(Arrays.asList(2, 3)));
```

78. Subsets

Medium | Topics | Companies

Given an integer array `nums` of **unique** elements, retu
all possible subsets (the power set).

The solution set **must not** contain duplicate subsets.
Return the solution in **any order**.

Example 1:

```
Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],
[2,3],[1,2,3]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

- `1 <= nums.length <= 10`

19K | 230 | 328

Java | Au

```
1  class S
2      pub
   nums) {
3      |
   ArrayLi
4      |
5      }
6  }
```

Saved

Testcase

If `nums = [1,2,3]`, subsets are:

```css
[]
[1]
[2]
[1,2]
[3]
[1,3]
[2,3]
[1,2,3]
```

Key Observations

- For every element, you have **two choices**:
  - Include it
  - Exclude it
- If `n` elements → total subsets = 2^n
- Order doesn't matter.

🧠 Best Approach: Backtracking (DFS)

We build subsets **incrementally** and explore all choices.

Idea

- Start with an empty list
- At every index:
    - Add current subset to answer
    - Try including each remaining element
    - Backtrack (remove last added element)

Copy code

```java
import java.util.*;

class Solution {

    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(0, nums, new ArrayList<>(), res);
        return res;
    }

    private void backtrack(int index, int[] nums,
                           List<Integer> curr,
                           List<List<Integer>> res) {

        // Add current subset
        res.add(new ArrayList<>(curr));

        // Try all possibilities starting from index
        for (int i = index; i < nums.length; i++) {

            // Choose
            curr.add(nums[i]);

            // Explore
```

```
        // Add current subset
        res.add(new ArrayList<>(curr));

        // Try all possibilities starting from index
        for (int i = index; i < nums.length; i++) {

            // Choose
            curr.add(nums[i]);

            // Explore
            backtrack(i + 1, nums, curr, res);

            // Unchoose (backtrack)
            curr.remove(curr.size() - 1);
        }
    }
}
```

⏱️ Time Complexity

- Total subsets = 2^n
- Each subset copying costs up to **O(n)**

👉 **Time:** $O(n \times 2^n)$

💾 Space Complexity

- Recursion stack: $O(n)$
- Result storage: $O(n \times 2^n)$

👉 **Space:** $O(n \times 2^n)$

🧩 Alternative (Bitmask – Just for Knowledge)

🧠 Why `new ArrayList<>(curr)`?

Because `curr` changes later.

We must store a **copy**, not the reference.

❌ Wrong:

```java
res.add(curr);
```

✅ Correct:

```java
res.add(new ArrayList<>(curr));
```

leetcode

Subset Sums

Difficulty: **Medium** Accuracy: **72.55%** Submissions: **184K+** Points: **4**

Given a array **arr** of integers, return the sums of all subsets in the list. Return the sums in any order.

Examples:

Input: arr[] = [2, 3]
Output: [0, 2, 3, 5]
Explanation: When no elements are taken then Sum = 0. When only 2 is taken then Sum = 2. When only 3 is taken then Sum = 3. When elements 2 and 3 are taken then Sum = 2+3 = 5.

Input: arr[] = [1, 2, 1]
Output: [0, 1, 1, 2, 2, 3, 3, 4]
Explanation: The possible subset sums are 0 (no elements), 1 (either of the 1's), 2 (the element 2), and their combinations.

Input: arr[] = [5, 6, 7]
Output: [0, 5, 6, 7, 11, 12, 13, 18]
Explanation: The possible subset sums are 0 (no elements), 5, 6, 7, and their combinations.

Constraints:

$1 \leq arr.size() \leq 15$

$0 \leq arr[i] \leq 10^4$

gfg

Copy code

```java
class Solution {
    public ArrayList<Integer> subsetSums(int[] arr) {
        ArrayList<Integer> res = new ArrayList<>();
        backtrack(0, arr, 0, res);
        return res;
    }

    private void backtrack(int index, int[] arr, int sum, ArrayList<Integer> re

        // Base case: reached end
        if (index == arr.length) {
            res.add(sum);
            return;
        }

        // Include current element
        backtrack(index + 1, arr, sum + arr[index], res);

        // Exclude current element
        backtrack(index + 1, arr, sum, res);
    }
}
```

⏱️ Time Complexity

- Total subsets = 2^n
- Each subset → constant work

✅ **Time:** $O(2^n)$

✅ **Space:** $O(2^n)$ **(result) +** $O(n)$ **recursion stack**

🔹 3. Side-by-Side Comparison

Feature	List<Integer>	ArrayList<Integer>
Type	Interface	Class
Object creation	❌ No	✅ Yes
Flexibility	High	Low
Programming style	Recommended	Less flexible
Implementation fixed?	❌ No	✅ Yes

🔹 2. `ArrayList<Integer>` (Class)

java Copy code

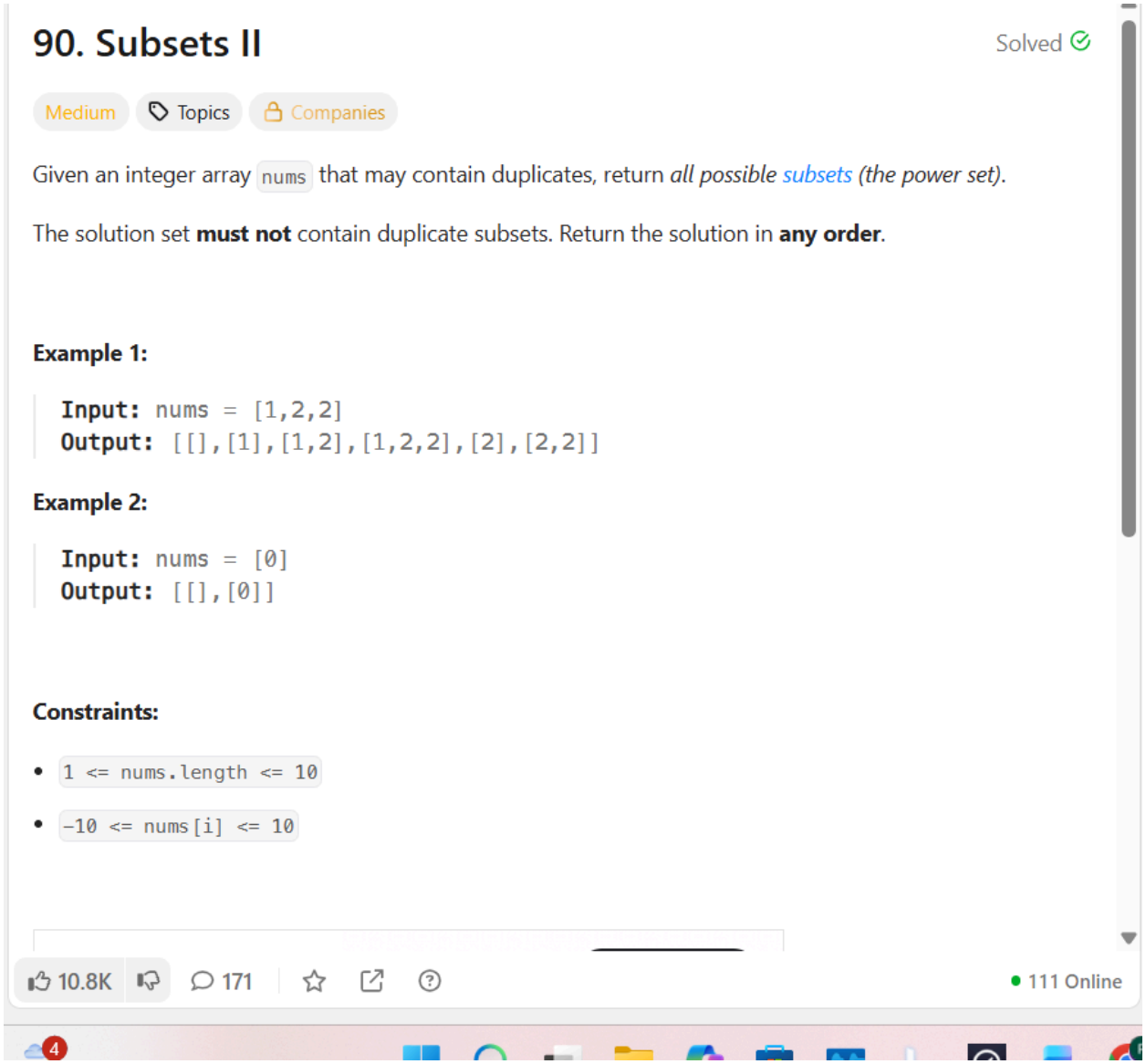
```java
ArrayList<Integer> list = new ArrayList<>();
```

What it is

- `ArrayList` is a **concrete class**
- Implements `List` interface

Key points

- Backed by a **dynamic array**
- Fast **random access** (`get(i)` → O(1))
- Slower insert/delete in middle



if you don't sort the array you will get wrong output

like this so sort array

1)i have used 1st sorting ,then a hashset to avoid duplicates,and adding it to res array

But this will increase our space complexity so it is not optimal and time complexity will also increase will using it

⏱️ Time Complexity (HashSet approach)

1️⃣ Generating subsets

- Total subsets generated = `2^n`

2️⃣ Inserting into HashSet

For each subset:

- HashSet needs to:
  - Compute hash of the list → **O(k)**
    (`k` = size of subset, worst case `k = n`)
  - Compare equality if collision → **O(k)**

Worst case per insertion:

```scss
O(n)
```

✅ Total Time Complexity

```scss
O(2^n * n)
```

⚠️ Extra hidden cost

- Creating new `ArrayList<>(curr)` → **O(n)**

So practically:

```scss
O(n * 2^n)
```

🧠 Space Complexity (HashSet approach)

1️⃣ HashSet storage

- Stores **all unique subsets**
- Worst case (all elements distinct):

```
2^n subsets
```

Each subset can take up to `O(n)` space.

```ini
Space = O(n * 2^n)
```

2️⃣ Recursion stack

```scss
O(n)
```

✅ Total Space Complexity

```scss
O(n * 2^n)
```

Copy code

🔄 Comparison with Optimal (Skip-Duplicates) Approach

Approach	Time	Space	Notes
HashSet-based	`O(n · 2^n)`	`O(n · 2^n)`	Extra hashing + memory
Sorting + skip duplicates	`O(n · 2^n)`	`O(n)` (excluding output)	Most efficient

> ⚠️ Output space (`res`) is always `O(n · 2^n)` and **not counted** in auxiliary space.

so use optimal

Copy code

```java
class Solution {
    public List<List<Integer>> subsetsWithDup(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        Arrays.sort(nums);   // 🔑 very important
        backtrack(0, nums, new ArrayList<>(), res);
        return res;
    }

    private void backtrack(int index, int[] nums, List<Integer> curr, List<List<I
        // Always add a COPY of current subset
        res.add(new ArrayList<>(curr));

        for (int i = index; i < nums.length; i++) {

            // 🔑 Skip duplicates at the same recursion level
            if (i > index && nums[i] == nums[i - 1]) {
                continue;
            }

            // choose
            curr.add(nums[i]);

            // explore
            backtrack(i + 1, nums, curr, res);
```

```java
private void backtrack(int index, int[] nums, List<Integer> curr, List<List<I
    // Always add a COPY of current subset
    res.add(new ArrayList<>(curr));

    for (int i = index; i < nums.length; i++) {

        // 🔑 Skip duplicates at the same recursion level
        if (i > index && nums[i] == nums[i - 1]) {
            continue;
        }

        // choose
        curr.add(nums[i]);

        // explore
        backtrack(i + 1, nums, curr, res);

        // unchoose
        curr.remove(curr.size() - 1);
    }
}
```

🧮 Time & Space Complexity

Time:

```scss
O(2^n)
```

Copy code

Space:

```scss
O(n) recursion depth
```

Copy code

leetcode chatgpt

39. Combination Sum

Medium | Topics | Companies

Given an array of **distinct** integers `candidates` and a target integer `target`, return *a list of all* ***unique combinations*** *of* `candidates` *where the chosen numbers sum to* `target`. You may return the combinations in **any order**.

The **same** number may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to `target` is less than `150` combinations for the given input.

Example 1:

```
Input: candidates = [2,3,6,7], target = 7
Output: [[2,2,3],[7]]
Explanation:
2 and 3 are candidates, and 2 + 2 + 3 = 7. Note that 2 can be used multiple
times.
7 is a candidate, and 7 = 7.
These are the only two combinations.
```

Example 2:

```
Input: candidates = [2,3,5], target = 8
```

20.7K 217 288 Online

leetcode

4️⃣ Correct approach: Backtracking (Standard Solution)

✅ Key ideas

- Use **DFS / Backtracking**
- Maintain an index so elements don't go backward
- Try all possibilities
- Stop when sum exceeds target

chatgpt

Java Auto

```java
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        List<List<Integer>> res= new ArrayList<>();
        backtrack(0,candidates,target,new ArrayList<>(),res);
        return res;
    }
    private void backtrack(int index ,int[] candidates,int target,List<Integer> curr,List<List<Integer>> res){
if(target==0){
    res.add(new ArrayList<>(curr));
    return;
}
//target negative kb hoga  jb hmara sum jyada ho then we will remove last added element and will try different index
if(target<0)return;
for(int i=index;i<candidates.length;i++){
    curr.add(candidates[i]);

    backtrack(i,candidates,target-candidates[i],curr,res);
    curr.remove(curr.size()-1);
}
    }
}
```

🧠 ROOT CAUSE (IMPORTANT)

🔴 `n` is NOT visible inside `backtrack()`

Your method is:

```java
private void backtrack(int row, char[][] board, List<List<String>> res)
```

But `n` is declared **only here** 👇

```java
public List<List<String>> solveNQueens(int n)
```

➡️ In Java, **method parameters are local**
➡️ `backtrack()` does NOT know what `n` is

That's why the compiler says:

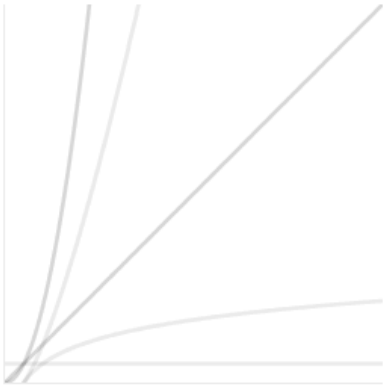
> ❌ cannot find symbol: variable n

---

of my below code time and space complexity are

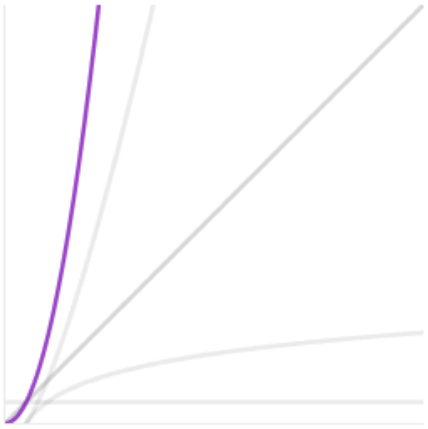
Time Complexity

$$O(N!)$$



Space Complexity

$O(N^2)$



51. N-Queens

Solved

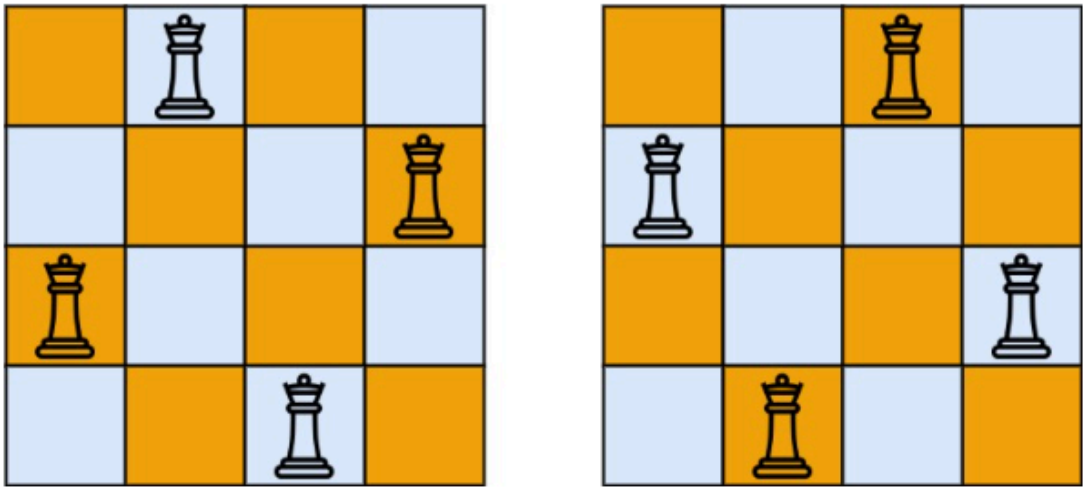
Hard | Topics | Companies

The **n-queens** puzzle is the problem of placing `n` queens on an `n x n` chessboard such that no two queens attack each other.

Given an integer `n`, return *all distinct solutions to the **n-queens puzzle***. You may return the answer in **any order**.

Each solution contains a distinct board configuration of the n-queens' placement, where `'Q'` and `'.'` both indicate a queen and an empty space, respectively.

Example 1:



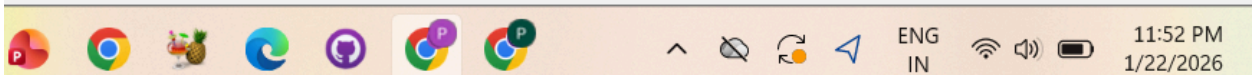
leetcode

```java
class Solution {
    public List<List<String>> solveNQueens(int n) {
        List<List<String>> res = new ArrayList<>();
        char[][] board = new char[n][n];
        for(int i=0;i<n;i++){
            Arrays.fill(board[i],'.');
        }
        backtrack(0,board,res);
        return res;
    }
    private void backtrack(int row,char[][] board,List<List<String>> res){
        int n = board.length;
        if(row==n){
        res.add(constructboard(board));

        return;
    }
for(int column=0;column<n;column++){

    if(isSafe(board,row,column)){
        board[row][column] ='Q';
        backtrack(row+1,board,res);
        //koi aur combination try krega
        board[row][column]='.';
    }
}
    }
    private boolean isSafe(char[][] board,int row,int column){
```

Saved

Ln 13, Col 18

Testcase | Test Result



ENG
IN

11:52 PM
1/22/2026

```java
    private boolean isSafe(char[][] board,int row,int column){
        int n = board.length;
        //checking column
        for(int r=0;r<row;r++){
            if(board[r][column]=='Q')return false;
        }
        //left daigonal
        for(int i=row, j=column;i>=0&&j>=0;i--,j--){
            if(board[i][j]=='Q')return false;
        }
        //right diagonal
        for(int i=row, j=column;i>=0&&j<n;i--,j++){
            if(board[i][j]=='Q')return false;
        }
        return true;
    }
    private List<String> constructboard(char[][] board){
        List<String> str = new ArrayList<>();
        for(char[] s:board){
            str.add(new String(s));
        }
        return str;
    }
}
```

```java
class Solution {

    boolean[] cols,leftdiagonal,rightdiagonal;
    public List<List<String>> solveNQueens(int n) {
         List<List<String>> res = new ArrayList<>();
        char[][] board = new char[n][n];
        cols = new boolean[n];
        //left diagonal row-col =constant
        leftdiagonal = new boolean[2*n-1];
        //right diagonal row+col=cons
        rightdiagonal = new boolean[2*n-1];
        for(int i=0;i<n;i++){
            Arrays.fill(board[i],'.');
        }
        backtrack(0,board,res);
        return res;
    }
    private void backtrack(int row,char[][] board,List<List<String>> res){
        int n = board.length;
        if(row==board.length){
            res.add(constructboard(board));
```

```java
private void backtrack(int row,char[][] board,List<List<String>> res){
    int n = board.length;
    if(row==board.length){
        res.add(constructboard(board));
        return ;
    }
    for(int c=0;c<board.length;c++){
        int d1=row-c+n-1;//shifted leftdiagonal by n-1 so that index w
        int d2 = row+c;
        if(cols[c] || leftdiagonal[d1] || rightdiagonal[d2]){
            continue;//means we can't put queen in this col so try ano
        }
        board[row][c] ='Q';
        cols[c]=leftdiagonal[d1]=rightdiagonal[d2]=true;
        backtrack(row+1,board,res);
         board[row][c] ='.';
        cols[c]=leftdiagonal[d1]=rightdiagonal[d2]=false;
            }
}
private List<String> constructboard(char[][] board){
    List<String> str = new ArrayList<>();
    for(char[] ch:board){
```

```java
private List<String> constructboard(char[][] boa
    List<String> str = new ArrayList<>();
    for(char[] ch:board){
        str.add(new String(ch));
    }
    return str;
}
```

Description | Editorial | Solutions | Submissions

37. Sudoku Solver

Solved

Hard | Topics | Companies | Hint

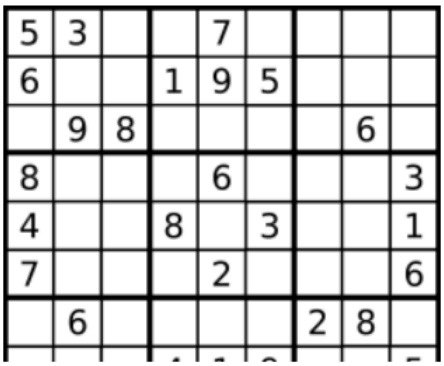
Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy **all of the following rules**:

1. Each of the digits `1-9` must occur exactly once in each row.
2. Each of the digits `1-9` must occur exactly once in each column.
3. Each of the digits `1-9` must occur exactly once in each of the 9 `3x3` sub-boxes of the grid.

The `'.'` character indicates empty cells.

Example 1:



10.9K | 267 | 108 Onlin

🧩 Problem Recap (In Simple Words)

You are given a **9×9 Sudoku board**:

- Digits `'1'` to `'9'` are fixed
- `'.'` means empty cell

Rules:

1. Each number **1–9 appears once per row**
2. Each number **1–9 appears once per column**
3. Each number **1–9 appears once per 3×3 box**

👉 Fill the board **in-place**.

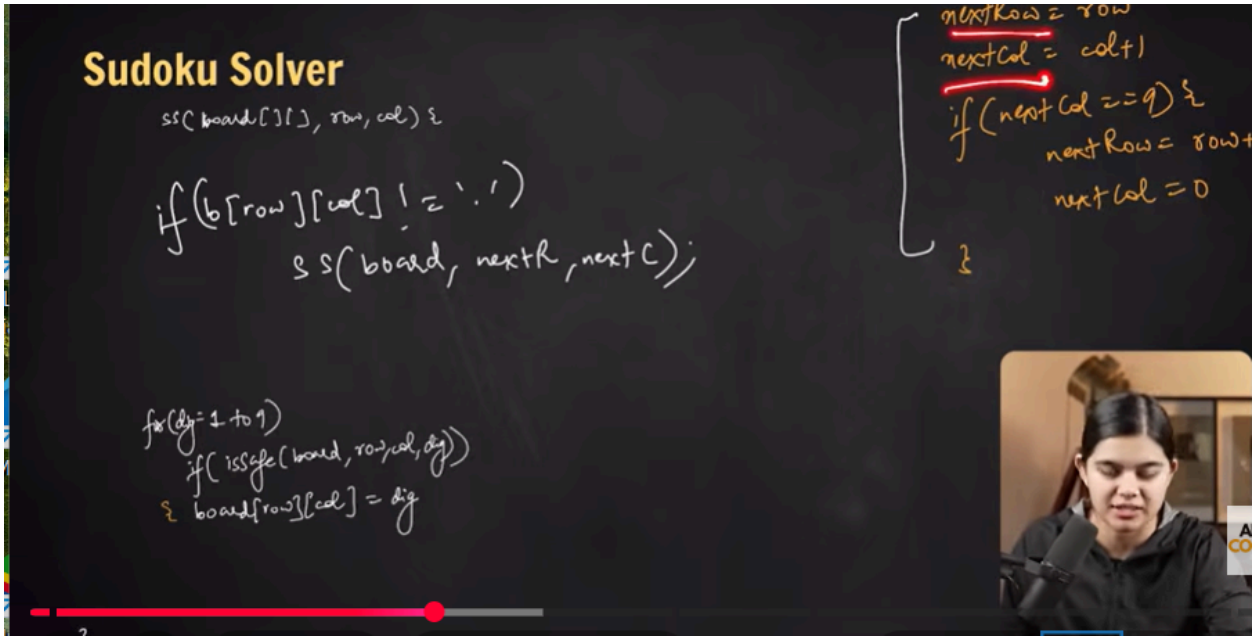
🔑 CORE IDEA: BACKTRACKING

Same philosophy as **N-Queens**:

> Try → Check → Recurse → Undo

Strategy:

- Find the **first empty cell**
- Try digits `'1'` to `'9'`
- If valid → place it
- Recurse to next empty cell
- If stuck → **backtrack**



Example (very clear)

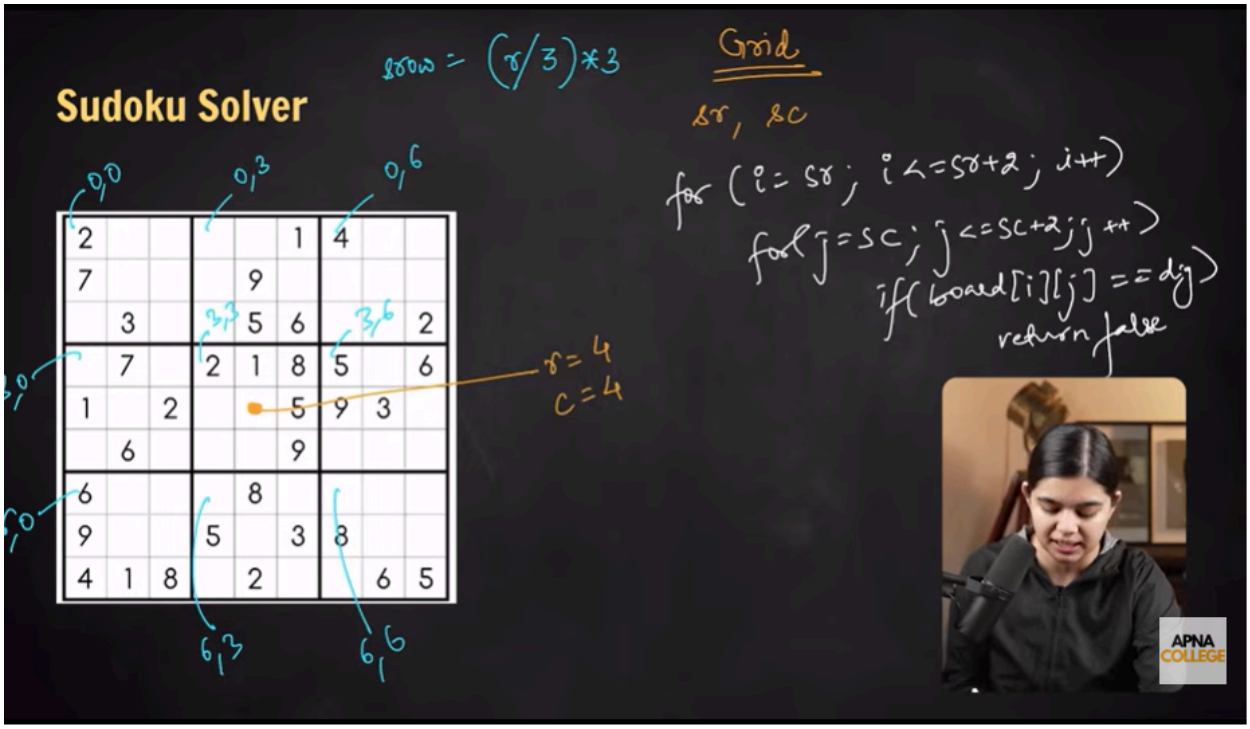
java Copy code

```java
class Demo {

    private void secret() {
        System.out.println("Private method");
    }

    public void show() {
        secret();   // ✅ allowed (same class)
    }
}
```

From another class ❌

java Copy code

```java
class Test {
    public static void main(String[] args) {
        Demo d = new Demo();
        d.secret();   // ❌ ERROR: secret() has private access
    }
}
```




```java
class Solution {
    private boolean helper(char[][] board,int row,int col){
        if(row==9)return true;
        int nextcol =col+1;
        int nextrow =row;
        if(nextcol ==9){
            nextcol=0;
            nextrow =row+1;
        }
        if(board[row][col] !='.'){
                return helper(board,nextrow,nextcol);
            }
            //placing digit
        for(char dig='1';dig<='9';dig++){
            if(isSafe(board,row,col,dig)){
                board[row][col] = dig;
                if(helper(board,nextrow,nextcol)){
                    return true;
                }
                //backtrack
                board[row][col]='.';
            }
        }
        return false;
    }
    private boolean isSafe(char[][] board,int row,int col,char dig){
//horizontal check row check
for(int i=0;i<9;i++){
```

ved

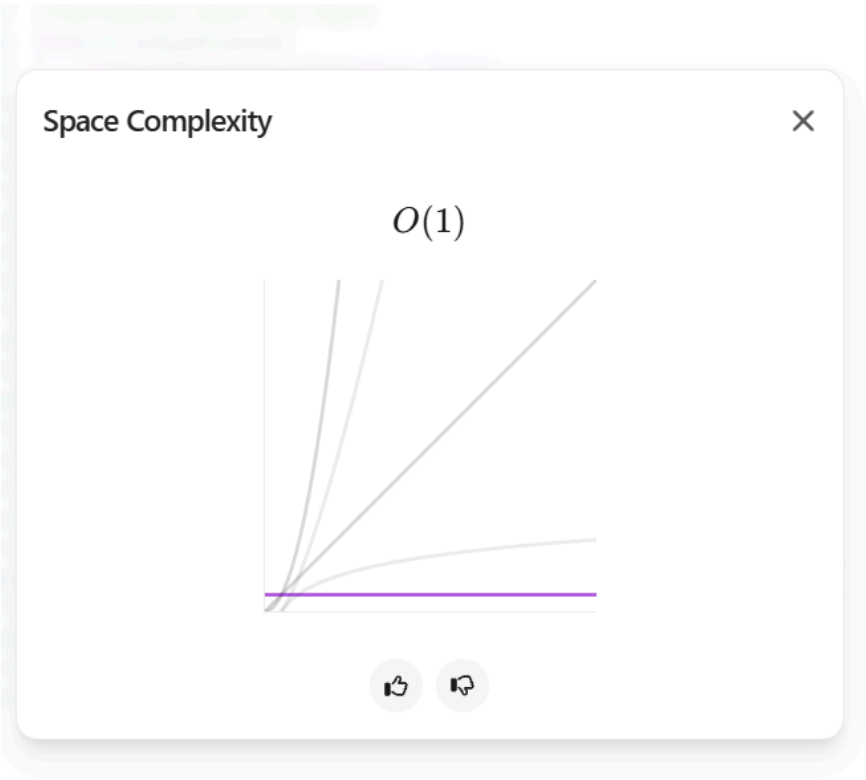
```java
//horizontal check row check
for(int i=0;i<9;i++){
    if(board[row][i]==dig)return false;
    //col check
    if(board[i][col]==dig)return false;


}
//grid check
    int sr = (row/3)*3;//starting row
    int sc =(col/3)*3;
    for(int i=sr;i<sr+3;i++){
        for(int j=sc;j<sc+3;j++){
            if(board[i][j]==dig)return false;
        }
    }
    return true;


    }
    public void solveSudoku(char[][] board) {
        helper(board,0,0);
    }
}
```

leetcode see apna college soln chatgpt

Time Complexity

$$O(9^{(}M))WhereMIsTheNumberOfEmptyCells$$

V



see chatgpt link for time complexity

🔁 How deep can recursion go in your code?

Key observation:

👉 You make **one recursive call per cell**, moving **left to right, top to bottom**.

```java
helper(board, nextrow, nextcol);
```

Maximum depth = number of cells visited

Sudoku board size:

```
9 × 9 = 81 cells
```

Even if all cells are empty, recursion goes **cell by cell**.

✅ MAXIMUM RECURSION DEPTH

Copy code

Ask anything

Maximum depth = number of cells visited

Sudoku board size:

```
9 × 9 = 81 cells
```

Even if all cells are empty, recursion goes **cell by cell**.

✅ MAXIMUM RECURSION DEPTH

```
81 calls
```

So recursion stack space:

```scss
O(81) ≈ O(1)
```

(h

Ask anything

Problem | Editorial | Submissions | Comments

M-Coloring Problem

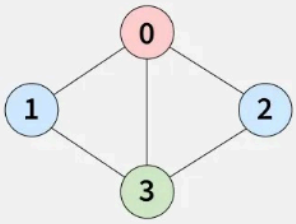
Difficulty: **Medium** Accuracy: **34.42%** Submissions: **181K+** Points: **4** Average Time: **45m**

You are given an undirected graph consisting of **V** vertices and **E** edges represented by a list **edges[][]**, along with an integer **m**. Your task is to determine whether it is possible to **color the graph** using at most **m** different colors such that no two adjacent vertices share the **same color**. Return true if the grap can be colored with at most **m** colors, otherwise return false.

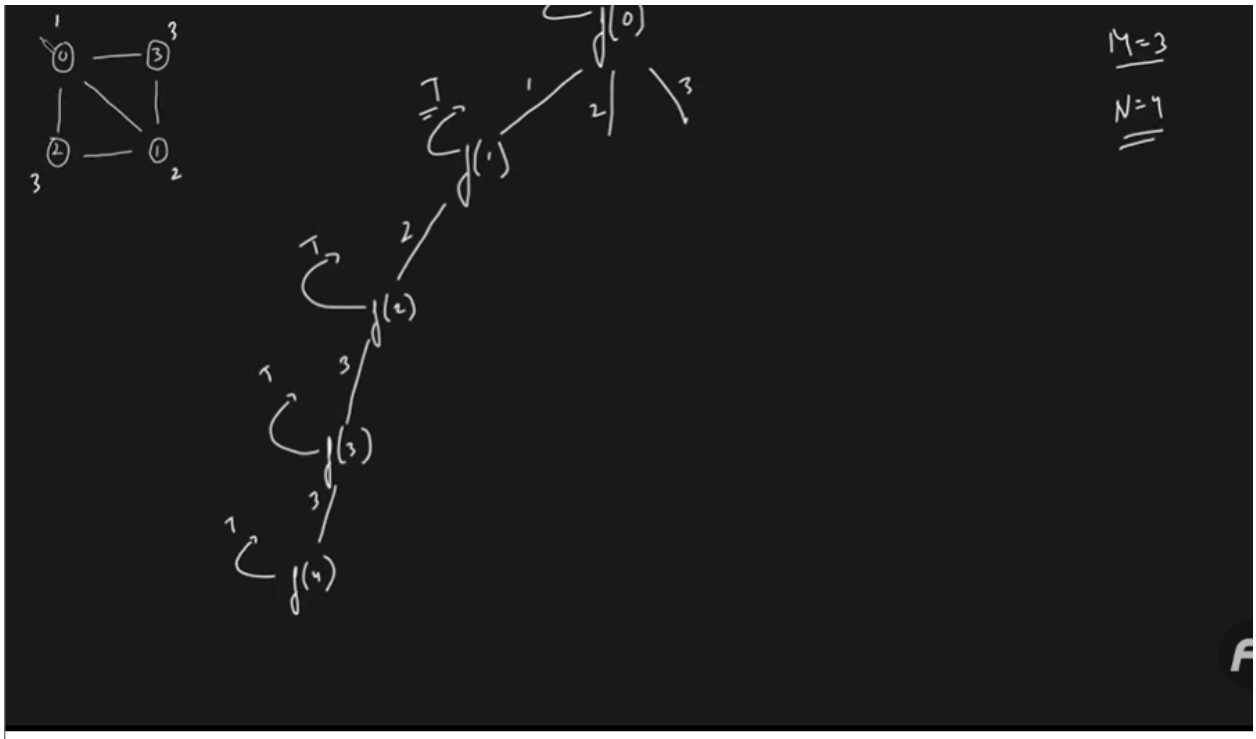
Note: The graph is indexed with 0-based indexing.

Examples:

Input: V = 4, edges[][] = [[0, 1], [1, 3], [2, 3], [3, 0], [0, 2]], m = 3
Output: true
Explanation: It is possible to color the given graph using 3 colors, for example, one of the possible ways vertices can be colored as follows:



Menu



L16. M-Coloring Problem | Backtracking

take U forward

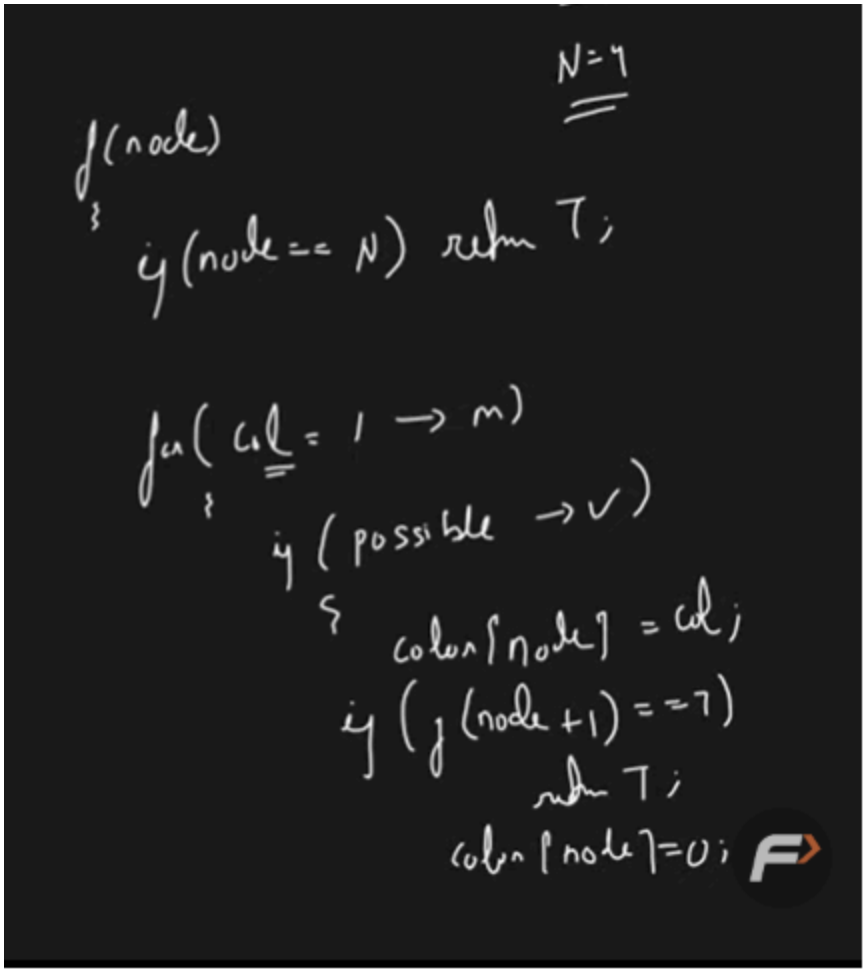
982K subscribers

Join

Subscribe

7.3K

Share



M-C

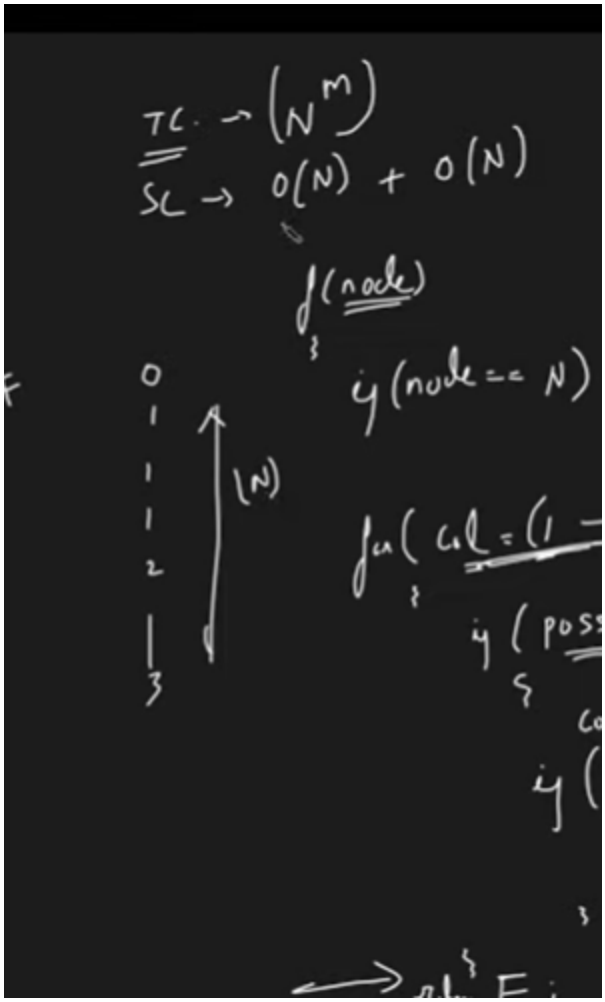
Diffi

You
alor
mos
can

Not

Exa

In
Ou
Ex
pc



🧠 Key Idea (Backtracking)

1. Try to color vertices **one by one**
2. For each vertex, try **all colors from 1 to m**
3. Before assigning a color, **check if it's safe**
4. If stuck → **backtrack**

This is similar to **N-Queens / Sudoku style recursion**.

🧩 Step 1: Build adjacency list

🧩 Step 1: Build adjacency list

Edges are given, but we convert them into a graph for easy traversal.

🧩 Step 2: Safety Check

A color is **safe** for a node if **none of its neighbors** already have that color.

🧩 Step 3: Recursive Coloring

- Base case: all vertices colored → return `true`
- Try all colors for current vertex
- If a color works → move to next vertex
- If none works → backtrack

```java
class Solution {
    boolean graphColoring(int v, int[][] edges, int m) {
        // building adjacent list
        List<List<Integer>> graph = new ArrayList<>();//vertex ke size ka list
        for(int i=0;i<v;i++){
            graph.add(new ArrayList<>());
        }
        for(int[] e:edges){
          graph.get(e[0]).add(e[1]);
graph.get(e[1]).add(e[0]);
        }
        int[] color = new int[v];
        return solve(0,graph,color,m,v);

    }
    private boolean solve(int node,List<List<Integer>> graph,int[] color,int m,int v){
        if(node==v)return true;
        //try all colors
        for(int c=1;c<=m;c++){
            if(isSafe(node,graph,color,c)){
                color[node]=c;
                if(solve(node+1,graph,color,m,v)){
                    return true;

                }
                //backtrack
                    color[node]=0;
            }
        }
        return false;
    }
    private boolean isSafe(int node,List<List<Integer>> graph,int[] color,int c){
        for(int neigh:graph.get(node)){
            if(color[neigh]==c)return false;
        }
        return true;
    }
```

🤔 Why does this warning appear?

This line causes it:

```java
List<Integer>[] graph = new ArrayList[V];
```

Java **does not fully support generic arrays**, so it warns you.

This is **VERY common** in graph problems.

Problem Editorial Submissions Comments

Rat in a Maze

Difficulty: **Medium** Accuracy: **35.75%** Submissions: **398K+** Points: **4** Average Time: **25m**

Consider a rat placed at position (0, 0) in an **n x n** square matrix **maze[][]**. The rat's goal is to reach the destination at position (n-1, n-1). The rat can move in four possible directions: **'U'(up)**, **'D'(down)**, **'L' (left)**, **'R' (right)**.

The matrix contains only two possible values:

- 0: A blocked cell through which the rat cannot travel.
- 1: A free cell that the rat can pass through.

Your task is to find **all possible paths** the rat can take to reach the destination, starting from (0, 0) and ending at (n-1, n-1), under the condition that the rat cannot **revisit** any cell along the same path. Furthermore, the rat can only move to adjacent cells that are within the bounds of the matrix and not blocked.
If no path exists, return an **empty list.**

Note: Return the final result vector in **lexicographically smallest order**.

Examples:

Input: maze[][] = [[1, 0, 0, 0], [1, 1, 0, 1], [1, 1, 0, 0], [0, 1, 1, 1]]
Output: ["DDRDRR", "DRDDRR"]
Explanation: The rat can reach the destination at (3, 3) from (0, 0) by two paths - DRDDRR and DDRDRR, when printed in sorted order we get DDRDRR DRDDRR.

gfg

🧠 ROOT CAUSE (Very Important)

❗ The problem statement says:

> **Return paths in lexicographically smallest order**

But your DFS explores directions in this order:

```
U → D → L → R
```

Lexicographical order for this problem is:

```
D → L → R → U
```

Because:

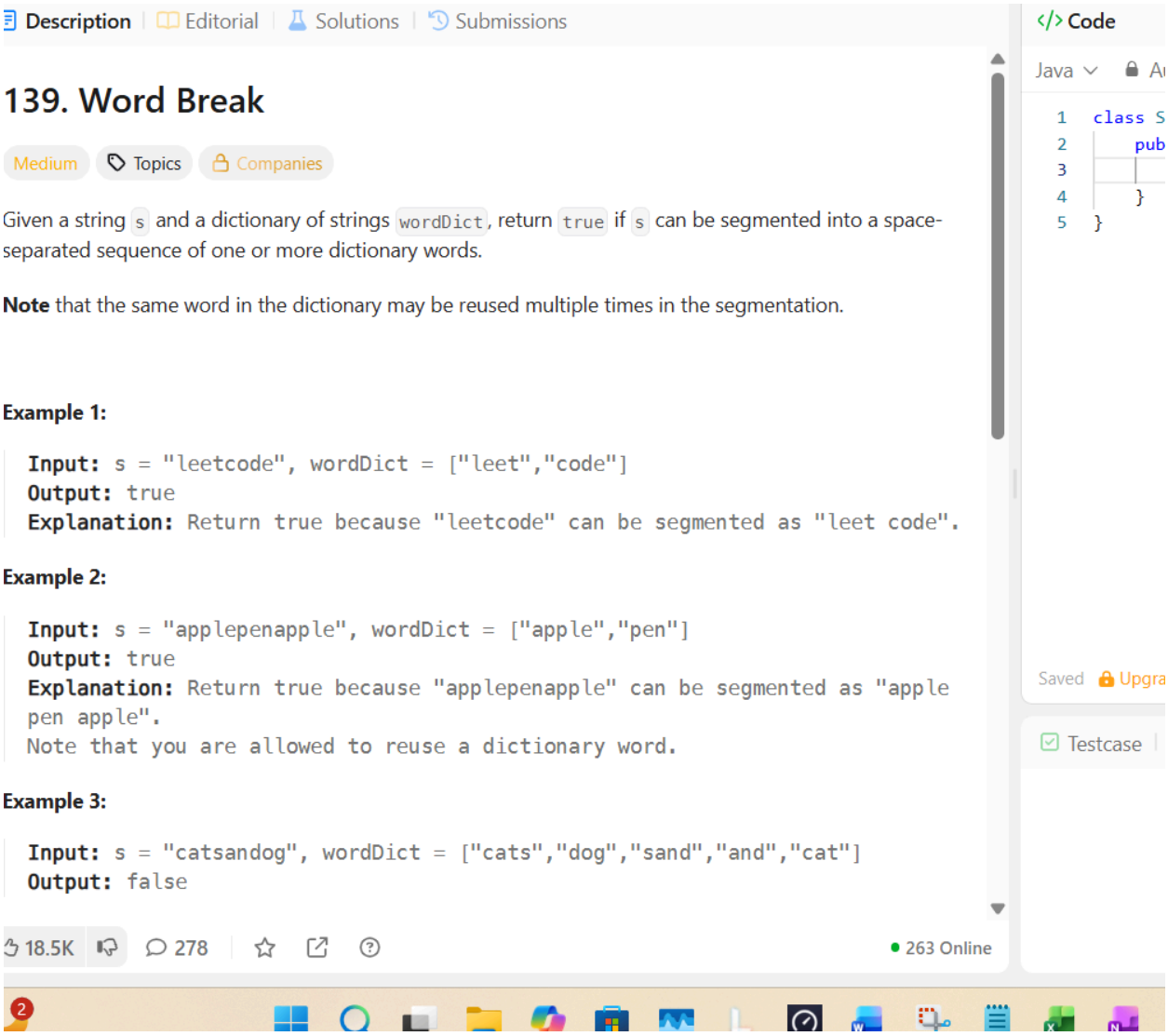
```
"D" < "L" < "R" < "U"
```

So even if paths are correct, **wrong traversal order = wrong output order**.

```java
class Solution {
    public ArrayList<String> ratInMaze(int[][] maze) {
        // code here
        int n = maze.length;
        boolean[][] visited = new boolean[n][n];
        ArrayList<String> res = new ArrayList<>();
       if(maze[0][0]==0 || maze[n-1][n-1]==0 )return res;
        backtrack(0,0,n,visited,maze,res,"");
        Collections.sort(res);
        return res;
    }
    private void backtrack(int row,int col,int n,boolean[][] visited,int[][] maze,ArrayList<String> res,String path){

        if(row==n-1 && col==n-1){
            res.add(path);
            return;


        }
         visited[row][col]=true;

        //up
        if(row-1>=0 && !visited[row-1][col] && maze[row-1][col]==1){
            backtrack(row-1,col,n,visited,maze,res,path+"U");
        }
        //down
        if(row+1<n && !visited[row+1][col] && maze[row+1][col]==1){
            backtrack(row+1,col,n,visited,maze,res,path+"D");
        }
        //left
        if(col-1>=0 && !visited[row][col-1] && maze[row][col-1]==1){
            backtrack(row,col-1,n,visited,maze,res,path+"L");
        }
        //right
        if(col+1<n && !visited[row][col+1] && maze[row][col+1]==1){
            backtrack(row,col+1,n,visited,maze,res,path+"R");
        }
```

```java
        int n = maze.length;
        boolean[][] visited = new boolean[n][n];
        ArrayList<String> res = new ArrayList<>();
       if(maze[0][0]==0 || maze[n-1][n-1]==0 )return res;
        backtrack(0,0,n,visited,maze,res,"");
        Collections.sort(res);
        return res;
    }
    private void backtrack(int row,int col,int n,boolean[][] visited,int[][] maze,ArrayList<String> res,String path){

        if(row==n-1 && col==n-1){
            res.add(path);
            return;


        }
         visited[row][col]=true;

        //up
        if(row-1>=0 && !visited[row-1][col] && maze[row-1][col]==1){
            backtrack(row-1,col,n,visited,maze,res,path+"U");
        }
        //down
        if(row+1<n && !visited[row+1][col] && maze[row+1][col]==1){
            backtrack(row+1,col,n,visited,maze,res,path+"D");
        }
        //left
        if(col-1>=0 && !visited[row][col-1] && maze[row][col-1]==1){
            backtrack(row,col-1,n,visited,maze,res,path+"L");
        }
        //right
        if(col+1<n && !visited[row][col+1] && maze[row][col+1]==1){
            backtrack(row,col+1,n,visited,maze,res,path+"R");
        }
        visited[row][col]= false;//backtrack
    }
}
```

Custom Input | Compile & Run | Submit


Description
Editorial
Solutions
Submissions
Code
Java
139. Word Break
Medium
Topics
Companies
Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of one or more dictionary words.
Note that the same word in the dictionary may be reused multiple times in the segmentation.
Example 1:
Input: s = "leetcode", wordDict = ["leet","code"]
Output: true
Explanation: Return true because "leetcode" can be segmented as "leet code".
Example 2:
Input: s = "applepenapple", wordDict = ["apple","pen"]
Output: true
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".
Note that you are allowed to reuse a dictionary word.
Example 3:
Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: false
18.5K
278
263 Online
Saved
Testcase

leetcode

1️⃣ Brute Force (Recursion / Backtracking)

💡 Idea

- Try to break the string at **every possible position**
- If the **prefix** exists in the dictionary, recursively solve for the **remaining suffix**
- If at any point the full string is consumed → `true`

This explores **all possible partitions**.

❌ Why it's slow

- Same substrings are solved **again and again**
- Exponential time

🧠 Brute Force Code (Java – Recursion)

Let's break `s.substring(0, i)` **slowly, with examples**.

📌 What does `substring(start, end)` mean in Java?

java Copy code

```
s.substring(start, end)
```

👉 It returns characters from **index** `start` **to** `end - 1`

👉 `start` **is included,** `end` **is excluded**

Time & Space Complexity (Brute)

- Time: O(2^n) (worst case – all partitions)

- Space: O(n) recursion stack

Copy code

```
s = "abcdef"
```

Each recursive call removes **one character**:

scss Copy code

```
solve("abcdef")
→ solve("bcdef")
→ solve("cdef")
→ solve("def")
→ solve("ef")
→ solve("f")
→ solve("")
```

➡️ Maximum depth = `n`

📦 Memory Used

- Each call stores:
  - local variables
  - return address

➡️ Stack grows linearly

🔢 Think in terms of cut positions

For a string of length `n`:

```less
a | b | c | d | e
```

There are `n-1` possible cut positions.

At each position:

- cut
- or don't cut

So total possibilities:

```scss
2^(n-1) ≈ 2^n
```

```java
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        HashSet<String> mp = new HashSet<>(wordDict);
      return  solve(s,mp);
    }
    private boolean solve(String s,HashSet<String> mp){
        if(s.length()==0)return true;
        for(int i=1;i<=s.length();i++){
            //trying all prefix
            String prefix = s.substring(0,i);
            if(mp.contains(prefix)){

               if(solve(s.substring(i),mp)){return true ;
            }
        }

    }
    return false;
    }
}
```

View less

this is the brute code which has time limited exceede

⚠️ Why Not $O(2^n)$ Anymore?

Brute Force:

- Same substrings recomputed again and again
- Exponential recursion tree

Memoization:

- Each substring solved once
- Recursion tree collapses into linear states

👉 **Exponential → Polynomial**

Step 4️⃣ Worst Case Input Explanation (Your Example)

ini

Copy code

```ini
s = "aaaaaaaab"
dict = ["a","aa","aaa","aaaa",...]
```

- Every prefix is valid
- All paths lead to `"b"`
- `"b"` is evaluated **once**
- Every other call instantly reuses memo

So instead of:

Copy code

```
100000+ calls ❌
```

You get:

nginx

Copy code

```nginx
n calls ✅
```

optimised one

```java
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        HashSet<String> mp = new HashSet<>(wordDict);
        HashMap<String,Boolean> memo = new HashMap<>();
      return  solve(s,mp,memo);
    }
    private boolean solve(String s,HashSet<String> mp,  HashMap<String,Boolean> memo ){
        if(s.length()==0)return true;
        if(memo.containsKey(s))return memo.get(s);
        for(int i=1;i<=s.length();i++){
            //trying all prefix
            String prefix = s.substring(0,i);
            if(mp.contains(prefix)){

              if(solve(s.substring(i),mp,memo)){
             memo.put(s,true);
               return true ;
            }
        }

    }
    memo.put(s,false);
    return false;
    }
}
```

Saved Ln 25

but it has timecomplexity n^2 can be close o n^3and space complexity n

🚨 Problem with your current solution

Your memoized code is **logically optimal**, but has two hidden costs:

1️⃣ Using `substring()` as state

- Creates new strings repeatedly
- Java `substring()` is **O(length)** (since Java 7)

2️⃣ Memo key = whole substring

- Extra memory
- Extra string creation

👉 This can push practical runtime closer to **O(n^3)**.

more optimised will be using dp

🔁 DP Transition Rule

java

Copy code

```
dp[i] = true if there exists j < i such that:
    dp[j] == true
    AND
    s.substring(j, i) ∈ wordDict
```

🧩 DP Code (Reference)

java

Copy code

```java
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        Set<String> dict = new HashSet<>(wordDict);
        int n = s.length();

        boolean[] dp = new boolean[n + 1];
        dp[0] = true;

        for (int i = 1; i <= n; i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && dict.contains(s.substring(j, i))) {
                    dp[i] = true;
                    break;
                }
            }
        }
        return dp[n];
    }
}
```

⏱️ Complexity

- Time: $O(n^2)$
- Space: $O(n)$

chatgpt

More optimised one will be using trie

40. Combination Sum II

Solved

Medium | Topics | Companies

Given a collection of candidate numbers (`candidates`) and a target number (`target`), find all unique combinations in `candidates` where the candidate numbers sum to `target`.

Each number in `candidates` may only be used **once** in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

```
Input: candidates = [10,1,2,7,6,1,5], target = 8
Output:
[
[1,1,6],
[1,2,5],
[1,7],
[2,6]
]
```

Example 2:

```
Input: candidates = [2,5,2,1,2], target = 5
Output:
```

12.1K | 219 | 108 Online

Java

```
1  cla
2
   com
   tar
3
   Arr
4
5
   new
6
7
8
9
   car
   res
10
11
12
13
14
   ++)
15
   ==c
16
17
18
   tar
19
```

Saved

Testcas

```java
class Solution {
    public List<List<Integer>> combinationSum2(int[] candidates, int target) {
        List<List<Integer>> res = new ArrayList<>();
        Arrays.sort(candidates);
        backtrack(0,candidates,target,res,new ArrayList<>());

        return res;
    }
    private void backtrack(int index,int[] candidates,int target, List<List<Integer>> res,List<Integer> curr){
        if(target==0){
            res.add(new ArrayList<>(curr));
            return;}

        for(int i=index;i<candidates.length;i++){
            if(i>index && candidates[i]==candidates[i-1])continue;
          if(candidates[i]>target)break;
            curr.add(candidates[i]);
            backtrack(i+1,candidates,target-candidates[i],res,curr);
            //backtrack
            curr.remove(curr.size()-1);
        }

    }
}
```

chatgpt leetcode

131. Palindrome Partitioning

Medium | Topics | Companies

Given a string `s`, partition `s` such that every substring of the partition is a **palindrome**. Return *all possible palindrome partitioning of* `s`.

Example 1:

```
Input: s = "aab"
Output: [["a","a","b"],["aa","b"]]
```

Example 2:

```
Input: s = "a"
Output: [["a"]]
```

Constraints:

- `1 <= s.length <= 16`
- `s` contains only lowercase English letters.

leetcode

```java
class Solution {
    public List<List<String>> partition(String s) {
        List<List<String>> res = new ArrayList<>();
        backtrack(0, s, new ArrayList<>(), res);
        return res;
    }

    private void backtrack(int start, String s,
                           List<String> curr,
                           List<List<String>> res) {

        // base case
        if (start == s.length()) {
            res.add(new ArrayList<>(curr));
            return;
        }

        // try all substrings starting from `start`
        for (int end = start + 1; end <= s.length(); end++) {
            String part = s.substring(start, end);

            if (isPalindrome(part)) {
                curr.add(part);
                backtrack(end, s, curr, res);
                curr.remove(curr.size() - 1); // backtrack
            }
        }
    }
```

va Auto

```
        // base case
        if (start == s.length()) {
            res.add(new ArrayList<>(curr));
            return;
        }

        // try all substrings starting from `start`
        for (int end = start + 1; end <= s.length(); end++) {
            String part = s.substring(start, end);

            if (isPalindrome(part)) {
                curr.add(part);
                backtrack(end, s, curr, res);
                curr.remove(curr.size() - 1); // backtrack
            }
        }
    }

    private boolean isPalindrome(String s) {
        int l = 0, r = s.length() - 1;
        while (l < r) {
            if (s.charAt(l++) != s.charAt(r--)) return false;
        }
        return true;
    }
}
```

ved Ln 2,

Testcase | Test Result

⏱️ Time & Space Complexity (Interview Gold)

Time:

- Worst case: `O(N * 2^N)`
  (every partition + palindrome check)

Space:

- Recursion depth: `O(N)`
- Result storage: `O(N * 2^N)`

🎯 How to explain to interviewer (1-liner)

> "We use backtracking. At each index, we try all palindromic prefixes and recu

you can optimise it using dpchatgpt

60. Permutation Sequence

Hard | Topics | Companies

The set `[1, 2, 3, ..., n]` contains a total of `n!` unique permutations.

By listing and labeling all of the permutations in order, we get the following sequence for `n = 3`:

1. `"123"`
2. `"132"`
3. `"213"`
4. `"231"`
5. `"312"`
6. `"321"`

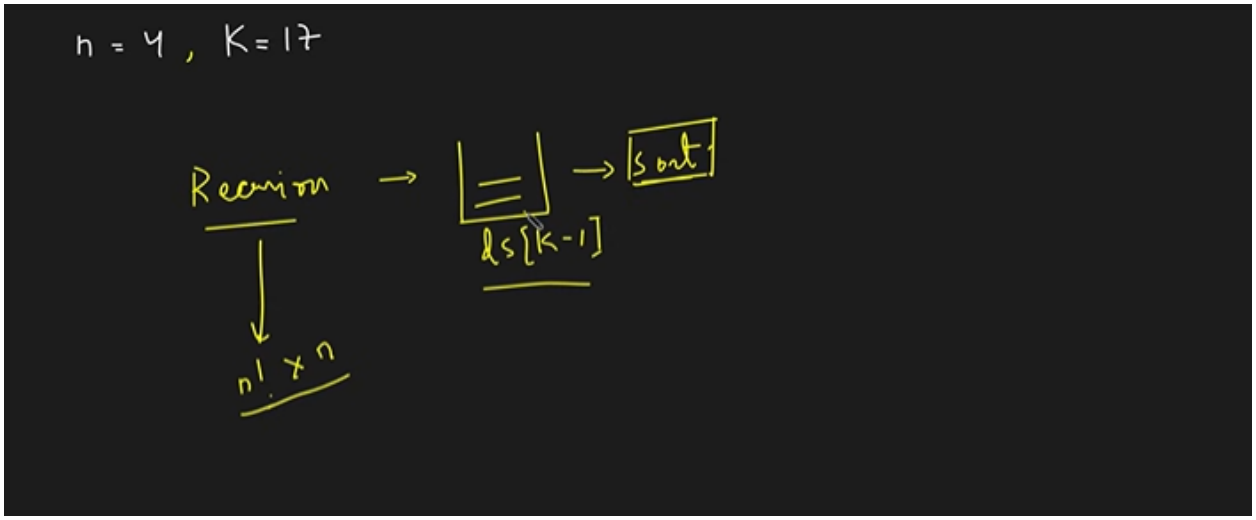
Given `n` and `k`, return the `k^th` permutation sequence.

Example 1:

```
Input: n = 3, k = 3
Output: "213"
```

Example 2:

```
Input: n = 4, k = 9
```



📘 What is Lexicographical Order?

Lexicographical order means **dictionary order**
(just like words are arranged in an English dictionary).

👉 We compare elements **from left to right**.

🧠 Simple Example (Words)

bash Copy code

```
apple
apply
bat
ball
cat
```

Why?

- `"apple"` comes before `"apply"` (l < y)
- `"bat"` comes before `"cat"` (b < c)

🧠 Brute Force Idea (Straightforward Thinking)

👉 "k-th permutation" means:

1. Generate **all permutations** of numbers `1..n`
2. Store them in **lexicographical order**
3. Return the **k-th one**

This is the **most natural approach** before optimization.

```java
class Solution {
    String ans ="";
    int count =0;
    public String getPermutation(int n, int k) {
        boolean[] used = new boolean[n+1];
        backtrack(n,k,used,new StringBuilder());
        return ans;
    }
    private void backtrack(int n,int k,boolean[] used,StringBuilder curr){
if(curr.length()==n){
    count++;//one full permutation formed
    if(count==k){
        ans = curr.toString();

    }
    return;
}
for(int i=1;i<=n;i++){
    if(used[i])continue;
    used[i]=true;
    curr.append(i);
    backtrack(n,k,used,curr);
    //if k has reached so we will stop backtracking
    if(!ans.equals(""))return;
    //backtrack
    used[i]=false;
    curr.deleteCharAt(curr.length()-1);
}
```

👨‍💼 Interview-Ready Explanation (Short & Clean)

> "The brute force approach generates permutations using backtracking.
> There are n! permutations, and building each permutation takes O(n) time, giving a total time complexity of O(n × n!).
> The space complexity is O(n) due to the recursion stack and auxiliary arrays."

🔥 This is a **model answer**.

🧾 Brute Force Recap (What the code does)

- Uses **backtracking**
- Generates permutations of `1..n`
- Counts them in **lexicographical order**
- Stops when the **k-th permutation** is found

⏱️ Time Complexity

🔹 How many permutations exist?

For `n` numbers:

```java
Total permutations = n!
```

🔹 Work done per permutation

- Building a permutation of length `n`
- Appending characters to `StringBuilder`

➡️ **O(n)** per permutation

🔹 Total Time Complexity

Worst case (when k = n!):

```scss
O(n × n!)
```

Even with **early stopping**, worst-case remains the same.

🔹 Why early stopping doesn't change Big-O?

Because:

- If `k = n!`, we still generate **all permutations**
- Big-O considers the **worst case**

But practically, early stopping **reduces runtime** for small k.

✅ Final Time Complexity (Say this)

> **Time Complexity: O(n × n!)**

🧠 Space Complexity

🔹 Recursive call stack

- Maximum depth = `n`

➡️ `O(n)`

🔹 Auxiliary structures

- `boolean[] used` → size `n`
- `StringBuilder` → max length `n`

➡️ `O(n)`

🔹 Total Space Complexity

scss Copy code

```
O(n)
```

Ask anything

optimal soln

✅ Problem Reminder

Given `n` and `k`, return the **k-th permutation (1-indexed)** of numbers `1…n` in **lexicographical order**.

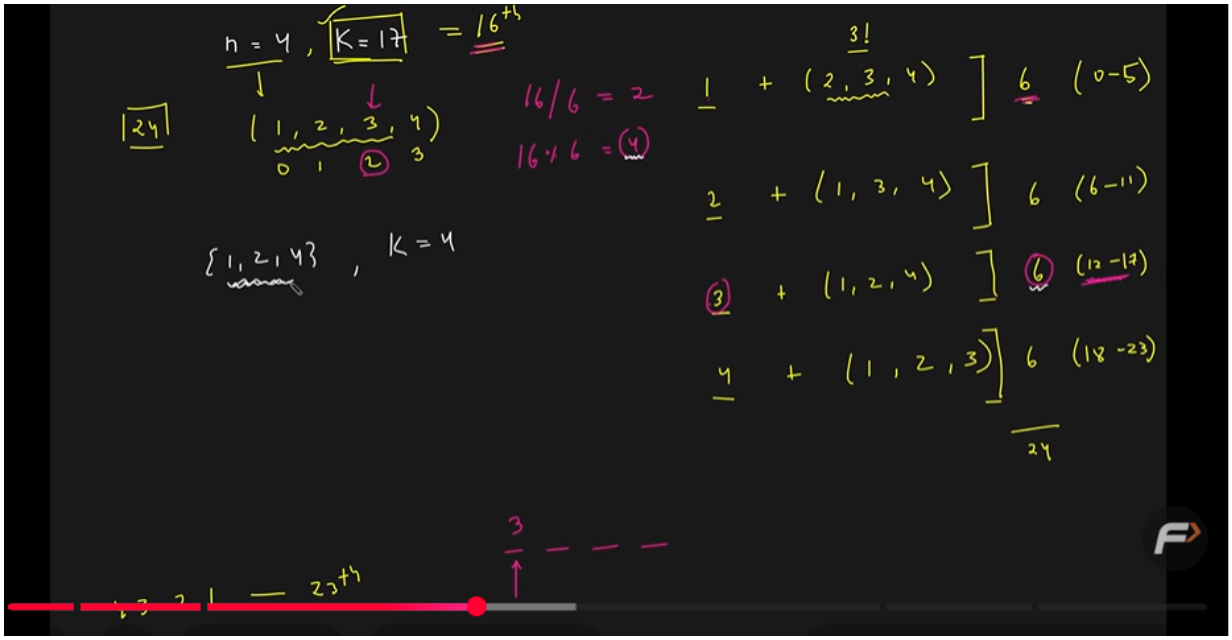
🧠 Key Insight (Why Optimal Works)

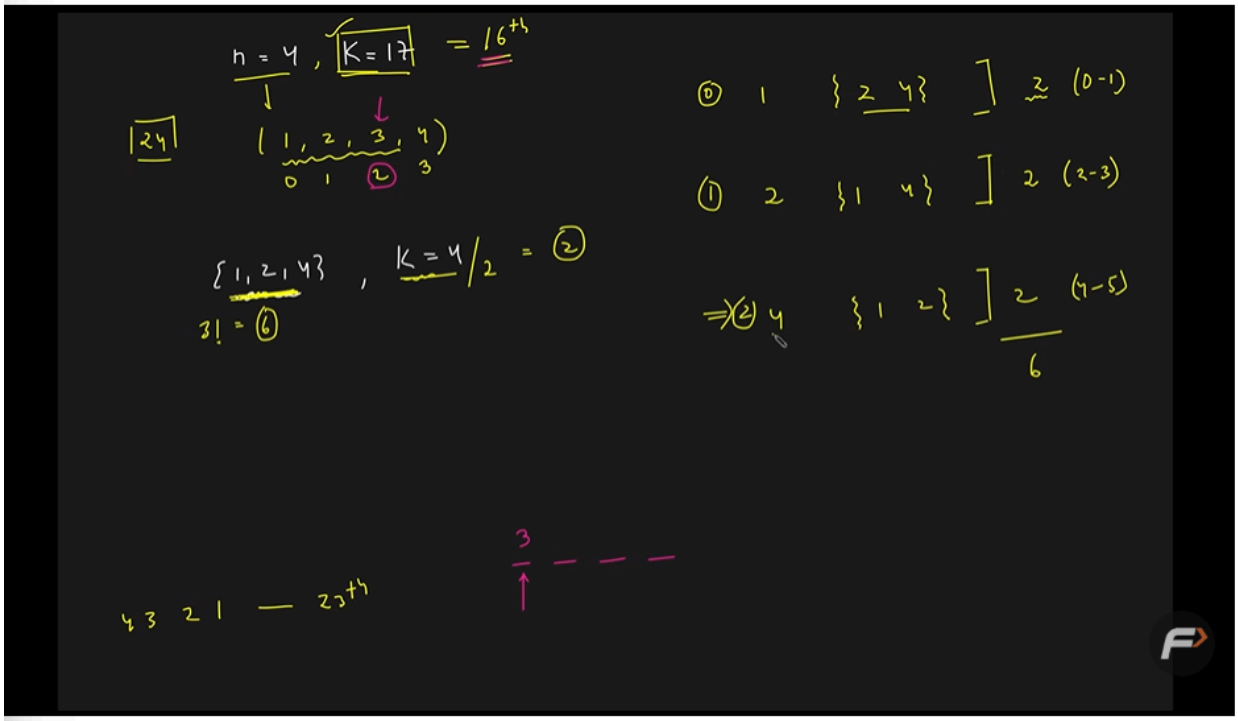
- Fixing the **first digit**, the remaining permutations are `(n-1)!`
- So permutations are grouped in **blocks of size** `(n-1)!`
- We can **skip entire blocks** instead of generating permutations one by one

This is called the **factorial number system**.

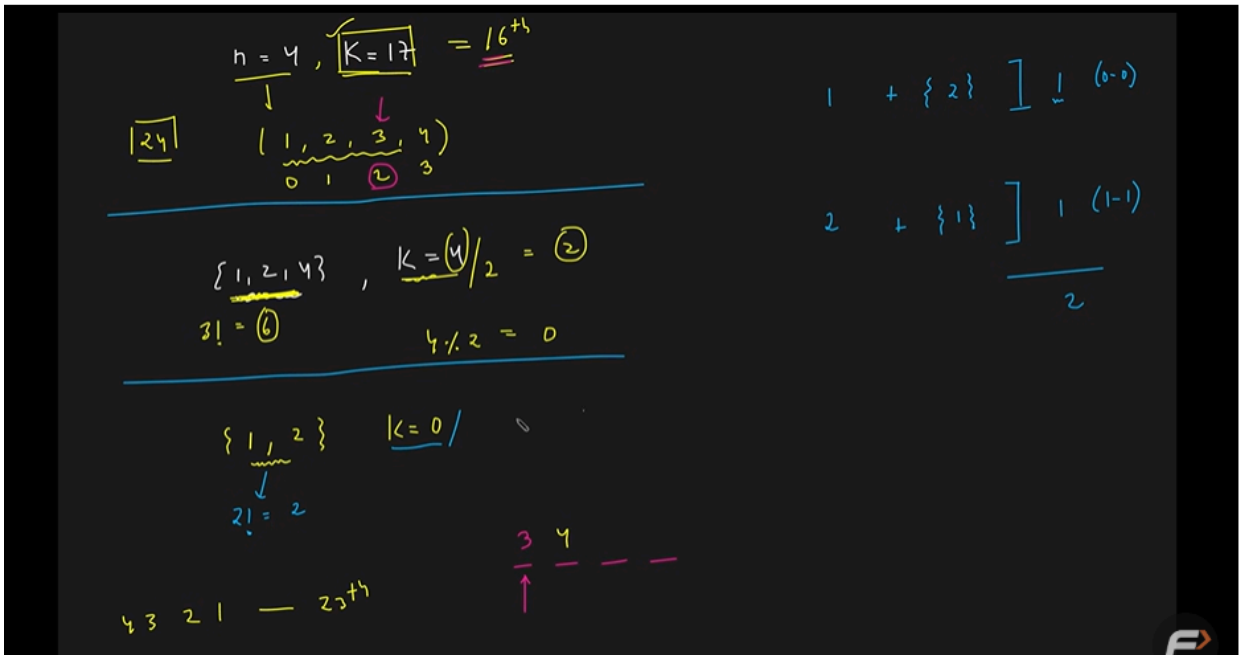
🧩 Step-by-Step Algorithm

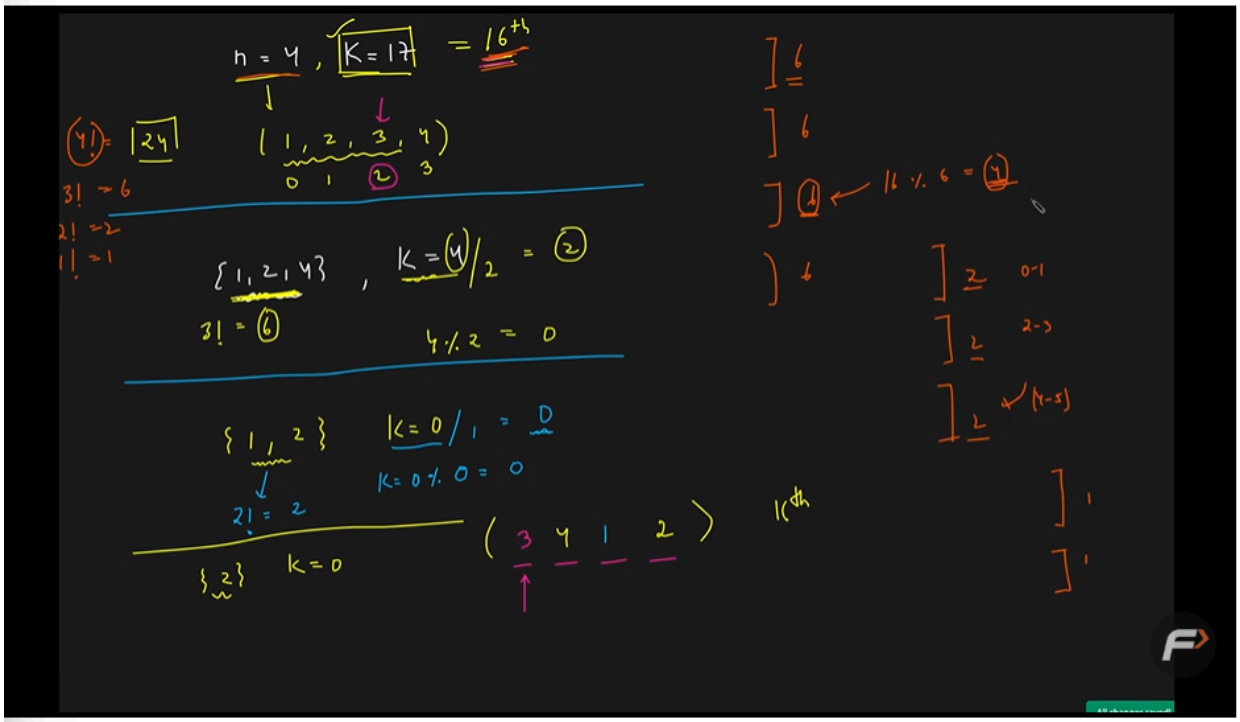
1. Store numbers `1…n` in a list
2. Precompute factorials
3. Convert `k` to **0-based** → `k--`
4. For each position:
   - `index = k / fact`
   - pick `numbers[index]`
   - remove it
   - `k = k % fact`
5. Continue until list is empty





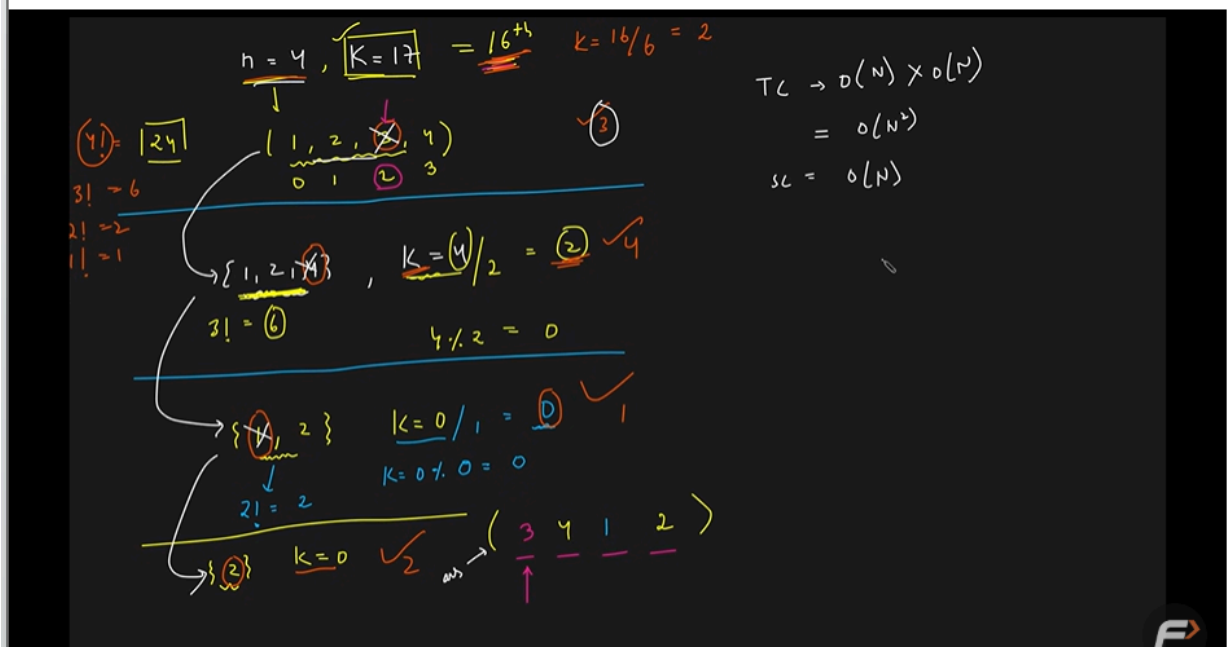
L18. K-th Permutation Sequence | Leetcode





L18. K-th Permutation Sequence | Leetcode

take U forward


n = 4 , K = 17 = 16th
k = 16/6 = 2
(4!) = 24
3! = 6
2! = 2
1! = 1
(1, 2, 3, 4)
0 1 2 3
3
{ 1, 2, 4 } , k = 4/2 = 2
4
3! = 6
4 % 2 = 0
{ 4, 2 } k = 0/1 = 0
1
k = 0 % 0 = 0
2! = 2
{ 2 } k = 0
2
ans → (3 4 1 2)
TC → O(N) X O(N)
= O(N²)
SC = O(N)

```java
class Solution {
    public String getPermutation(int n, int k) {
        List<Integer> num = new ArrayList<>();
        // we will store elem and calculate n-1fact
        int fact =1;
        for(int i=1;i<n;i++){
fact*=i;
num.add(i);
        }
        num.add(n);

        StringBuilder ans = new StringBuilder("");
        k-=1;
        while(true){
ans.append(num.get(k/fact));
num.remove(k/fact);//each removal costs n time complexity
if(num.size()==0){
    break;
}
k = k%fact;
fact = fact/num.size();
        }

        return ans.toString();
    }
```

🔍 Step-by-Step Time Analysis

1️⃣ Initial loop

```java
for(int i = 1; i < n; i++) {
    fact = fact * i;
    numbers.add(i);
}
numbers.add(n);
```

- Runs `n-1` times
- Each operation is **O(1)**

✅ **Time = O(n)**

2️⃣ while(true) loop

```java
while(true) {
    ans = ans + numbers.get(k / fact);
    numbers.remove(k / fact);
    if(numbers.size() == 0) break;

    k = k % fact;
    fact = fact / numbers.size();
}
```

This loop runs **n times** (one digit added per iteration).

3️⃣ Key costly operation

```java
numbers.remove(index);
```

`numbers` is an **ArrayList**.

- Removing by index shifts elements
- Worst case shift = `n` elements

⛔ **Time = O(n)** per removal

4️⃣ Total cost of while loop

- Loop runs `n` times
- Each time:
  - `get()` → O(1)
  - `remove()` → **O(n)** ❗

✅ Final Time Complexity

pgsql

```
O(n)  (initial loop)
+ O(n²) (while loop)
------------------
= O(n²)
```

🧠 Why This Is Still Optimal

- We **avoid generating n! permutations**
- n ≤ 9, so `O(n²)` is extremely fast
- This is the **best possible practical solution**

👨‍💼 Interview-Ready Explanation (Say This)

> "The while loop runs n times.
> Each iteration removes an element from an ArrayList, which costs O(n) due to shifting.
> Therefore, the total time complexity is $O(n^2)$."

use stingbuilder instead of string as it is muttable it will not vcreate a new string after adding a new element to it

2️⃣ String concatenation is expensive

In your code:

```java
ans = ans + numbers.get(k / fact);
```

Strings are **immutable** in Java.

So each concatenation:

- Creates a new string
- Copies old content

Cost over all iterations:

```mathematica
O(1 + 2 + 3 + ... + n) = O(n²)
```

This **extra hidden cost** hurts runtime score.

👉 Using `StringBuilder` improves this